Orientación a Objetos II 2026

Explicación de práctica
Semana del 30 de marzo

Staff de la práctica

JTPs

Agustín Ortu
Germán Ruiz
Juan Cruz Gardey
Leonardo Loza Bonora

Ayudantes

Diego De Sogos
Cristian Cerro
Nero Brindesi
Luciana Tanevitch
Felipe Dioguardi
Ignacio Traberg
Guadalupe Martin
Catalina Zuppa Moro
Martin Tau
Giuliana Maltempo

Adscriptos/colaboradores

Gomez Torres Rafael Genaro
Nathaniel Frutos
Camila Tarrio

Joaquin Tartaruga
Bautista Clua
Lautaro Gubia
Bianca Defelipe
Matheo Lamiral
Augusto Boschiazzo

Actividades prácticas

Explicación de práctica: 2 turnos

- **Lunes 18 a 19 hs**; Aula 5
- **Martes 8:30 a 9:30 hs;** Aula 10A

Consultas de práctica: 7 horarios

- Lunes 19:00 a 20:30 hs., aula 5 (aula sin pcs)
- Martes 10:00 a 11:30 hs., aula 1-3 (aula sin pcs)
- Martes 17:30 a 19:00 hs, aula 1-1
- Martes 19:00 a 20:30 hs., aula 1-1
- Miércoles 17:30 a 19:00 hs, aula 1-4 (aula sin pcs)
- Miércoles 19:00 a 20:30 hs., aula 2 (aula sin pcs) + aula 1-3 (aula sin pcs)
- Jueves 10:00 a 11:30 hs., sala de PC.

Refactoring

Trabajos Prácticos

Cuadernillo de Repaso

Cuadernillo de actividades - Refactoring

Guías y material de referencia

Catálogo de Bad Smells de Objetos 2

Este archivo contiene un catálogo con la clasificación

Orientación a Objetos 2

Cuadernillo Semestral de Actividades

-Refactoring -

Actualizado: 30 de marzo de 2026

El presente cuadernillo estará en elaboración durante el semestre y tendrá un compilado con todos los ejercicios que se usarán durante la asignatura respecto al tema Refactoring.

Refactoring

- Refactorización (sustantivo): cambio realizado en la estructura interna de un programa informático para que sea más fácil de entender y más “barato” de modificar sin cambiar su comportamiento observable.

- Refactorizar (verbo): reestructurar software aplicando una serie de refactorizaciones sin cambiar su comportamiento observable.

Refactoring v1 - Fowler

Refactoring

- Refactorizar NO es tirar el código y escribirlo de nuevo
- Se debe preservar el comportamiento
- Refactorizar es seguir un método para mejorarlo:

    1. Identificar code smells
    2. Determinar cómo mejorarlo
    3. Aplicar la mejora

Ejercicio 1.1

La clase `Cliente` tiene el siguiente protocolo. **¿Cómo puede mejorarlo?**

```
/**
* Retorna el límite de crédito del cliente
*/
public double lmtCrdt() {...

/**
* Retorna el monto facturado al cliente desde la fecha f1 a la fecha f2
*/
protected double mtFcE(LocalDate f1, LocalDate f2) {...

/**
* Retorna el monto cobrado al cliente desde la fecha f1 a la fecha f2
*/
private double mtCbE(LocalDate f1, LocalDate f2) {...
```

Ejercicio 1.3

```
public void imprimirValores() {
        int totalEdades = 0;
        double promedioEdades = 0;
        double totalSalarios = 0;

        for (Empleado empleado : personal) {
                totalEdades = totalEdades + empleado.getEdad();
                totalSalarios = totalSalarios + empleado.getSalario();
        }
        promedioEdades = totalEdades / personal.size();

        String message = String.format("El promedio de las edades es %s y el total de salarios es %s", promedioEdades, totalSalarios);

        System.out.println(message);
```

-Mal olor detectado en el código

-Extracto del código que presenta el mal olor

-Refactoring a aplicar que resuelve el mal olor

-Código con el refactoring aplicado

Ejercicio 1.3

```
public void imprimirValores() {
        int totalEdades = 0;
        double promedioEdades = 0;
        double totalSalarios = 0;

        for (Empleado empleado : personal) {
                totalEdades = totalEdades + empleado.getEdad();
                totalSalarios = totalSalarios + empleado.getSalario();
        }
        promedioEdades = totalEdades / personal.size();

        String message = String.format("El promedio de las edade
totalSalarios);
        System.out.println(message);
}
```

-Mal olor detectado en el código

-Extracto del código que presenta el mal olor

-Refactoring a aplicar que resuelve el mal olor

-Código con el refactoring aplicado

Code smell: long method
imperative loops
temporary field

Comenzamos por uno: long method

Refactoring a aplicar:

Replace temp with query
Extract method

Ejercicio 1.3

Procedimiento de Extract Method:

1. Crea un nuevo método y asígnale un nombre que refleje su finalidad.
2. Copia el código extraído del método de origen al nuevo método de destino.
3. Revisa el código extraído en busca de referencias a variables cuyo ámbito sea local al método de origen. Se trata de variables locales y parámetros del método.
4. Comprueba si hay variables temporales que se utilicen únicamente dentro de este código extraído. Si es así, declaralas en el método de destino como variables temporales... Puedes eliminar las variables temporales con «Replace Temp with Query».
5. Pasa al método de destino como parámetros las variables de ámbito local que se leen del código extraído.
6. Compila cuando hayas tratado todas las variables de ámbito local.
7. Reemplaza el código extraído del método de origen por una llamada al método de destino.
8. Compila y prueba.

Procedimiento extraido de Refactoring v1 - Fowler

Ejercicio 1.3

```
public void imprimirValores() {
        double promedioEdades = this.calcularPromedioEdades();
        double totalSalarios = 0;

        for (Empleado empleado : personal) {
                totalSalarios = totalSalarios + empleado.getSalario();
        }
totalS

}
```

```
public double calcularPromedioEdades() {
        int totalEdades = 0;
        double promedioEdades = 0;

        for (Empleado empleado : personal) {
                totalSalarios = totalSalarios + empleado.getEdad();
        }

        promedioEdades = totalEdades / personal.size();

        return promedioEdades;
}
```

-Mal olor detectado en el código

-Extracto del código que presenta el mal olor

-Refactoring a aplicar que resuelve el mal olor

-Código con el refactoring aplicado

Bad smell: long method
imperative loops
temporary field

menzamos por uno: long method

factoring a aplicar:

Replace temp with query

Extract method

Ejercicios 2 al 5:

Ejercicio 2: Iteradores circulares

Ejercicio 3: Iteradores circulares bis

Ejercicio 4: Alcance en Redes Sociales

Ejercicio 5: Productos

CharRing.java

```
1 public c
2   char[]
3   int idx
4
5   publi
6     cha
7     sou
8     src
9     res
```

Publicacion.java

```
1 package redesocial;
2 public class Publicacion {
3   private String texto;
4   private int likes;
5   public Publicacion(String texto) {
6     this.texto = texto;
7     this.likes = 0;
8   }
9   public void darLike() { likes++; }
```

- Presentan código a refactorizar, indicando número de línea.
- Indican qué refactoring se debe aplicar.
- Luego de resolverlos, revisar con algún ayudante.

Ejercicio 6

Para cada una de las siguientes situaciones, realice en forma iterativa los siguientes pasos:

1. indique el mal olor,
2. indique el refactoring que lo corrige,
3. aplique el refactoring, mostrando el resultado final (código y/o diseño según corresponda).

Si vuelve a encontrar un mal olor, retorne al paso (i).

Ejercicio 6.2

```java
public class Juego {
    // ......
    public void incrementar(Jugador j) {
        j.puntuacion = j.puntuacion + 100;
    }
    public void decrementar(Jugador j) {
        j.puntuacion = j.puntuacion - 50;
    }
}
```

```java
public class Jugador {
    public String nombre;
    public String apellido;
    public int puntuacion = 0;
}
```

(1) Indicar mal olor → (2) Indicar refactoring → (3) Aplicar refactoring

Repetir proceso si vuelve a encontrar un mal olor

Ejercicio 6.2

Procedimiento de Move Method:

1. Examina todas las propiedades utilizadas por el método de origen que estén definidas en la clase de origen. Valora si también deben trasladarse.
2. Comprueba si hay otras declaraciones del método en las subclases y superclases de la clase de origen.
3. **Declara el método en la clase de destino.**

…

```
public class Juego {
    // ......
    public void incrementar(Jugador j) {
        j.incrementar = j.puntuacion + 100;
    }
    public void decrementar(Jugador j) {
        j.puntuacion = j.puntuacion - 50;
    }
}
```

```
public class Jugador {
    public String nombre;
    public String apellido;
    public int puntuacion = 0;

    public void incrementar() {}
}
```

Ejercicio 6.2

4. Copie el código del método de origen al de destino. Adapte el método para que funcione en su nueva ubicación.
5. Compile la clase de destino.
6. **Convierte el método de origen en un método de delegación.**
7. Compila y prueba.

…

```
public class Juego {
    // ......
    public void incrementar(Jugador j) {
        j.incrementar();
    }
    public void decrementar(Jugador j) {
        j.puntuacion = j.puntuacion - 50;
    }
}
```

```
public class Jugador {
    public String nombre;
    public String apellido;
    public int puntuacion = 0;

    public void incrementar() {
        this.puntuacion = this.puntuacion + 100;
    }
}
```

Ejercicio 6.2

8. Decide si eliminas el método de origen o lo mantienes como método de delegación.

9. Si eliminas el método de origen, sustituye todas las referencias por referencias al método de destino.

10. Compila y prueba.

```
public class Juego {
    // ......
    public void decrementar(Jugador j) {
        j.puntuacion = j.puntuacion - 50;
    }
}
```

```
public class Jugador {
    public String nombre;
    public String apellido;
    public int puntuacion = 0;

    public void incrementar() {
        this.puntuacion = this.puntuacion + 100;
    }
}
```

¿Quedan malos olores?

¡A repetir el proceso!

Ejercicio 7

1. ¿Hay código duplicado?
Indique claramente en qué líneas se encuentra.

```
1   abstract class Etiqueta {
2       protected String nombreProducto;
3       protected double precio;
4
5       public Etiqueta(String nombre, double precio) {
6           this.nombreProducto = nombre;
7           this.precio = precio;
8       }
9   }
```

```
11  class EtiquetaSimple extends Etiqueta {
12      public EtiquetaSimple(String nombre, double precio) {
13          super(nombre, precio);
14      }
15
16      public void generar() {
17          System.out.println("--- ETIQUETA BÁSICA ---");
18          System.out.println("Producto: " + nombreProducto);
19          System.out.println("Precio: $" + precio);
20          System.out.println("----------------------");
21      }
22  }
```

```
24  class EtiquetaDetalle extends Etiqueta {
25      public EtiquetaDetalle(String nombre, double precio) {
26          super(nombre, precio);
27      }
28
29      public void generar() {
30          System.out.println("--- ETIQUETA DETALLE ---");
31          System.out.println("Producto: " + nombreProducto);
32          System.out.println("Precio sin imp.: $" + (precio * 0.79));
33          System.out.println("Precio final: $" + precio);
34          System.out.println("----------------------");
35      }
36  }
```

Ejercicio 7

2. Se quiere aplicar el refactoring *Pull Up Method* para subir el método generar() a la superclase Etiqueta.

¿Es posible hacerlo en el código anterior?

→ Precondiciones ←

```
1   abstract class Etiqueta {
2       protected String nombreProducto;
3       protected double precio;
4
5       public Etiqueta(String nombre, double precio) {
6           this.nombreProducto = nombre;
7           this.precio = precio;
8       }
9   }
```

```
11  class EtiquetaSimple extends Etiqueta {
12      public EtiquetaSimple(String nombre, double precio) {
13          super(nombre, precio);
14      }
15
16      public void generar() {
17          System.out.println("--- ETIQUETA BÁSICA ---");
18          System.out.println("Producto: " + nombreProducto);
19          System.out.println("Precio: $" + precio);
20          System.out.println("----------");
21      }
22  }
```

```
24  class EtiquetaDetalle extends Etiqueta {
25      public EtiquetaDetalle(String nombre, double precio) {
26          super(nombre, precio);
27      }
28
29      public void generar() {
30          System.out.println("--- ETIQUETA DETALLE ---");
31          System.out.println("Producto: " + nombreProducto);
32          System.out.println("Precio sin imp.: $" + (precio * 0.79));
33          System.out.println("Precio final: $" + precio);
34          System.out.println("----------");
35      }
36  }
```

Ejercicio 7

2. Se quiere aplicar el refactoring *Pull Up Method* para subir el método generar() a la superclase Etiqueta.

¿Es posible hacerlo en el código anterior? → Precondiciones ←

Procedimiento de Pull Up Method:

1. Revisa los métodos para asegurarte de que sean idénticos.
2. Si los métodos tienen firmas diferentes, cambia las firmas a la que quieras utilizar en la superclase.
3. Crea un nuevo método en la superclase, copia en él el cuerpo de uno de los métodos, ajústalo y compila.
4. Elimina un método de la subclase.
5. Compila y prueba.
6. Sigue eliminando métodos de la subclase y probando hasta que solo quede el método de la superclase.
7. Echa un vistazo a los llamantes de este método para ver si puedes cambiar un tipo requerido a la superclase.

Ejercicio 7

2. Se quiere aplicar el refactoring *Pull Up Method* para subir el método generar() a la superclase Etiqueta.

¿Es posible hacerlo en el código anterior? → Precondiciones ←

Procedimiento de Pull Up Method:

1. Revisa los métodos para asegurarte de que sean idénticos.
2. Si los métodos tienen firmas diferentes, cambia las firmas a la que quieras utilizar en la superclase.

…

¿Cómo logramos que los métodos sean idénticos, manteniendo la misma firma?

Ejercicio 7

3. Mencione los refactorings previos necesarios para que sea posible aplicar Pull Up Method.
4. Aplique Pull Up Method para subir el método generar() a la superclase Etiqueta.

```java
abstract class Etiqueta {
    protected String nombreProducto;
    protected double precio;

    public Etiqueta(String nombre, double precio) {
        this.nombreProducto = nombre;
        this.precio = precio;
    }
}
```

```java
class EtiquetaSimple extends Etiqueta {
    public EtiquetaSimple(String nombre, double precio) {
        super(nombre, precio);
    }

    public void generar() {
        System.out.println("--- ETIQUETA BÁSICA ---");
        System.out.println("Producto: " + nombreProducto);
        System.out.println("Precio: $" + precio);
        System.out.println("----------");
    }
}
```

```java
class EtiquetaDetalle extends Etiqueta {
    public EtiquetaDetalle(String nombre, double precio) {
        super(nombre, precio);
    }

    public void generar() {
        System.out.println("--- ETIQUETA DETALLE ---");
        System.out.println("Producto: " + nombreProducto);
        System.out.println("Precio sin imp.: $" + (precio * 0.79));
        System.out.println("Precio final: $" + precio);
        System.out.println("----------");
    }
}
```

Ejercicio 8

1. Enumere los code smell y que refactorings utilizará para solucionarlos.

2. Aplique los refactorings encontrados, mostrando el código refactorizado luego de aplicar cada uno.

3. **Analice el código original y detecte si existe un problema al calcular las estadísticas. Explique cuál es el error y en qué casos se da ¿El error identificado sigue presente luego de realizar los refactorings? En caso de que no esté presente, ¿en qué momento se resolvió? De acuerdo a lo visto en la teoría, ¿podemos considerar esto un refactoring?**

```
public class Document {
    List<String> words;

    public long characterCount() {
        long count = this.words
                .stream()
                .mapToLong(w -> w.length())
                .sum();
        return count;
        }
    public long calculateAvg() {
        long avgLength = this.words
                .stream()
                .mapToLong(w -> w.length())
                .sum() / this.words.size();
        return avgLength;
        }
// Resto del código que no importa
}
```

Ejercicio 8

1. Enumere los code smell y que refactorings utilizará para solucionarlos.

2. Aplique los refactorings encontrados, mostrando el código refactorizado luego de aplicar cada uno.

3. **Analice el código [illegible] este un prob[illegible] estadísticas. [illegible] error y en qué casos se [illegible] error identificado sigue presente luego de realizar los refactorings? En caso de que no esté presente, ¿en qué momento se resolvió? De acuerdo a lo visto en la teoría, ¿podemos considerar esto un refactoring?**

Se debe preservar el comportamiento

```
public class Document {
    List<String> words;

    public long ch[illegible]
                [illegible]ream()
                .mapToLong(w -> w.length())
                .sum();
        return count;
        }
    public long calculateAvg() {
        long avgLength = this.words
                .stream()
                .mapToLong(w -> w.length())
                .sum() / this.words.size();
        return avgLength;
        }
// Resto del código que no importa
}
```